10.1 记忆系统与知识管理

> **生成模型**：gpt-5.4 (openai/gpt-5.4)
> **Token 消耗**：输入 ~8k tokens，输出 ~3k tokens（估算）

到了第 10 章，OpenClaw 的基础能力你已经差不多都摸熟了。接下来真正决定“这东西能不能天天用”的，往往不是模型多强，而是它记不记得住事。

很多人第一次用 Agent 都会有一种挫败感：今天聊得挺顺，明天重新开一个会话，它又像失忆了一样。你明明昨天才说过“我主要写 TypeScript”“服务器在新加坡”“发给我的日报别太官话”，结果第二天它又从头问一遍。这个落差，其实不是 OpenClaw 特有的问题，而是大模型的基本限制：模型本身不会像人一样自动形成长期记忆，除非系统把信息另外存起来。

OpenClaw 在这件事上的思路很朴素：**把记忆落到文件里**。不是搞一堆你看不见摸不着的黑箱状态，而是把长期信息写进 `MEMORY.md` ，把日常记录写进 `memory/` 目录。这样做有一个很大的好处：你自己也能看，也能改，也能删。AI 不是“偷偷记住你”，而是“把它认为重要的东西记录下来，并在之后尽量找回来”。

10.1.1 OpenClaw 到底记住了什么

先别把“记忆”想得太玄。对 OpenClaw 来说，常见的记忆大概就四类：

- 你的长期偏好，比如常用语言、写作风格、常去的目录、常用命令。
- 稳定事实，比如服务器地址、项目代号、团队成员分工、常用 API 提供商。
- 正在持续推进的事情，比如这周在修哪个 bug，下个月准备上线什么。
- 不适合每次都重复说的协作规则，比如“改代码前先读相关文件”“回答尽量短一点”。

这里有个边界最好一开始就想清楚：**不是所有聊天内容都值得记**。你随口说一句“今天天气真热”，系统没必要长期保存；但你说“我所有 Docker 项目都放在 `~/srv` 下面”，这种信息以后反复会用到，就很适合进长期记忆。

所以，OpenClaw 的记忆更像一个“可检索的工作笔记本”，不是人格模拟器。它不是要假装自己跟你从小一起长大，而是尽量别让你在重复劳动上浪费时间。

10.1.2 `MEMORY.md`：长期记忆的总账本

`MEMORY.md` 可以理解成 Agent 的长期备忘录。只要你愿意，完全可以把它当成一份给 AI 写的“用户说明书”。

比如下面这些内容，就很适合放进去：

```
# 用户偏好
- 默认用中文回答，别太正式
- 写代码时优先 TypeScript / Python
- 提交信息尽量简短，别写空话

# 常用环境
- 主开发机：macOS
- 项目根目录常在 ~/Programming
- 常用部署方式：Docker Compose

# 长期项目
- 正在写《深入 OpenClaw》
- 第 I 部分面向刚上手的读者，强调可操作性
```

有些用户会问：这东西要我手写吗？答案是，**你可以手写，也可以让 Agent 帮你写，但最好定期自己看一眼**。因为只要是“记忆”，就会有过期的时候。你三个月前常用某个模型，不代表今天还在用；你去年常驻某个仓库，今年可能已经换了。

我的建议是，`MEMORY.md` 不要追求“全”，而要追求“稳”。写那些三周后、三个月后仍然大概率成立的事实。越稳定，价值越高。

10.1.3 `memory/` 目录：每天发生的事，放这里

如果说 `MEMORY.md` 是总账本，那 `memory/YYYY-MM-DD.md` 更像工作日记。它适合记当天发生的事，比如：

- 今天排查过什么问题。
- 某个任务做到哪一步了。
- 某个临时决策为什么这么定。
- 某个坑已经踩过一次，下次别再踩。

这种按天记录的方式很实用。原因很简单：很多信息不是永久有效，但在接下来几天特别有用。比如你今天刚把一个 Telegram 机器人重新配好了，明天再排故时，这条记录就很值钱；但半年后它可能就只

是历史资料。

一个比较自然的使用习惯是：

1. 长期稳定规则写进 `MEMORY.md` 。
2. 临时进展、排查过程、会议结论写进当天日志。
3. 周末或者阶段切换时，挑重要内容回收进 `MEMORY.md` 。

这样做，记忆系统就不会越用越乱。很多人把所有东西都往 `MEMORY.md` 塞，最后那份文件又长又杂，检索效果也会跟着变差。

10.1.4 AI 为什么“像是记得你”

从用户视角看，体验大概是这样的：你提了一个问题，OpenClaw 在回答前，会先看看当前上下文里够不够信息；不够的话，再去相关记忆文件里找。找到以后，它把这些片段重新带回这轮对话，于是你就会感觉它“记得”。

注意，这里不是魔法，也不是模型脑子里真存了一份“关于你的完整画像”。更准确地说，它做的是三步：

1. 你以前说过的重要信息，被写进了磁盘文件。
2. 现在你又提到相关话题，系统尝试把旧信息找出来。
3. 找到的片段被重新送进当前这次推理里。

所以它记忆的质量，其实取决于两件事：

- 之前有没有记对。
- 现在能不能找准。

前者靠你和系统一起维护，后者就要靠搜索机制了。

10.1.5 什么叫向量搜索，为什么它比关键词更像“会意”

如果只靠关键词搜索，会有个很常见的问题：你写的是“公网地址”，下次问的是“服务器 IP”，字面上没完全对上，普通检索就可能漏掉。

向量搜索可以把它理解成一种“按意思找相近内容”的办法。系统会先把一段文字转成一串数字，这串数字不重要，你完全不用背；重要的是，**意思接近的句子，转出来的位置也会更接近**。这样你搜“部署机器在哪”，它就不一定非得等你把原话一字不差地打出来。

如果你学过数据库，可以把它粗略理解成“不是按字面匹配，而是按语义距离排序”。如果你没学过，也没关系，就记一句人话版：

> 关键词搜索像在翻书目录，向量搜索更像你脑子里那个“我记得大概提过这事”的感觉。

当然，向量搜索也不是神。它会有误召回，也会漏召回。所以最稳的做法永远是：**记忆内容要写清楚，搜索只是帮你更容易找回来。**

10.1.6 嵌入提供商怎么配

向量搜索要先做一件事：把文本转成向量。这个步骤通常叫 embedding，中文常翻成“嵌入”。OpenClaw 支持多种提供商，你不用太纠结术语，把它当成“谁来负责把文字编码成可搜索的语义表示”就行。

一个典型配置长这样：

```
{
  agents: {
    defaults: {
      memorySearch: {
        provider: "openai"
      }
    }
  }
}
```

如果你已经在用 OpenAI，这往往是最省事的起点。除此之外，常见思路还有：

- 想少折腾，先用你已经有账号的云服务。
- 想统一账单，就选和主模型同一家或同一套平台。
- 想尽量本地化，就考虑本地嵌入模型。
- 想控制成本，就优先看嵌入模型的价格，而不是聊天模型的价格。

这里有个新手常犯的错：把“主模型供应商”和“嵌入供应商”混成一回事。其实它们可以一样，也可以不一样。你完全可以聊天走 Anthropic，记忆检索走 OpenAI 或本地嵌入服务。别把它想复杂，本质上只是两条不同的能力链。

10.1.7 怎么把记忆系统用顺手

如果你想让 OpenClaw 真的越用越懂你，下面这几条特别重要。

第一，告诉它什么值得记

不要指望系统百分百猜中你的意图。你真想让它记住，就明确说：“把这件事记下来。”这比模糊地提一嘴可靠得多。

第二，记事实，不记情绪噪音

“我最近有点烦”这种话当然可以聊，但不太适合进长期记忆。相反，“我正在准备春招，需要把算法题讲得更基础”就很有价值，因为它能直接影响后续协作方式。

第三，定期整理

建议每隔一两周看一次 `MEMORY.md` 。删掉过时信息，合并重复条目，把散落在每日日志里的重要内容往上提。你会发现，记忆系统和笔记系统一样，不整理就会发霉。

第四，别把密码往里塞

这个看起来像废话，但真的值得单独说。API Key、数据库密码、私钥、恢复码，这些都不该放进记忆文件。记忆的目标是提高协作效率，不是变成事故源。

第五，给内容起个能被搜到的说法

比如你可以写“生产服务器 / VPS / 部署机”这样的别名，或者在一条记录里把常见叫法一起放进去。这样无论你下次怎么问，都更容易召回。

10.1.8 你可以把它当成“给未来的自己留上下文”

有意思的是，OpenClaw 的记忆系统不只是给 AI 用，很多时候也在帮你自己。你回头翻 `MEMORY.md` 和每日日志，会很直观地看到：自己平常怎么工作、哪些事情老重复、哪些坑一直在踩。这其实已经有点像工程化的个人知识管理了。

所以更实用的理解方式是：OpenClaw 并不是单方面“记住你”，而是你和它一起维护了一套共享上下文。你写进去的，不只是让 AI 变聪明，也是让未来的协作成本变低。

本节小结

这一节最核心的点其实就三个。第一，OpenClaw 的长期记忆不是藏在黑箱里，而是落在 `MEMORY.md` 和 `memory/` 目录里，用户自己可读可改。第二，向量搜索的价值不在于“高级”，而在于它能按意思找

内容，让记忆召回更自然。第三，记忆系统要想长期好用，关键不是堆更多信息，而是把稳定事实、短期日志和定期整理这三件事做好。

当你把记忆维护顺手以后，OpenClaw 才会真正从“每次都要重新解释背景”的工具，变成一个能持续接住上下文的长期助手。